But that's not necessarily true. Concurrent tasks are coordinated in such a way that they overlap in their execution, but their execution may or may not not be done at the same point in time. We need parallelism to execute different tasks at the same point in time.

Parallelism is the ability to *execute* multiple tasks at the same time. Imagine that there are multiple CPU cores on a server. Parallelism allows those CPU cores to work at the same time to execute different tasks simultaneously. Multiple CPU cores are required for this to work because a single CPU core can only run one set of instructions at a time.

Parallelism provides advantages when it comes to system throughput and overall performance. A parallel system will be able to fully use its compute resources to serve requests. You often require less CPU to serve the same requests when adding parallelism to an application.

Concurrency is a property of programming languages, but the actual implementation of that language determines how parallelism is achieved. It's possible to have a concurrent language that isn't parallel, for example. In fact, you'll learn about this next.

Let's go into how Ruby's concurrency works.

Ruby's Concurrency Model

Ruby uses threads to achieve concurrency. In Ruby, `Thread.new` runs a block of code concurrently with other code in that process. Concurrent code can finish in any order. Here's a simple example that uses `Thread.new` to print out characters. If you run this code multiple times, you'll see that the print order changes.

```
irb> Thread.new{ puts "a" }; Thread.new{ puts "b" }
a
b

irb> Thread.new{ puts "a" }; Thread.new{ puts "b" }
b
a
```

Ruby threads share memory. This means that you can change a variable in one thread, and reading that variable in another thread will reflect the changed value. This isn't a bad thing by itself, but it leads to a whole class of programming bugs called race conditions.